Lab 4 Basic: Password Lock

Objective

In this lab, you will design a finite state machine (FSM) and use a Linear Feedback Shift Register (LFSR) to generate a pseudo-random password. You will then implement a password lock that verifies whether the input sequence entered using buttons matches the generated password.

This lab builds on your **Lab 4 Practice** and extends your skills in FSM design and hardware-based input handling.

Action Items

a. I/O list

I/O	Connected to	Definition
clk	W5	Clock signal with the frequency of 100MHZ.
rst	BTNC	Asynchronous active-high reset signal
btnL	BTNL	Seen as input 1 to the FSM.
btnR	BTNR	Seen as input 0 to the FSM.
btnD	BTND	Signal to shift LFSR in IDLE state. Signal to reset num[3:0] in PLAY state.
btnU	BTNU	Signal to advance FSM to next state.
out[1:0]	LED[8:7]	Signal to control LED[8:7]; Display match output.
num[3:0]	LED[15:12]	Signal to control LED[15:12]; Display user-entered sequence.
LED[3:0]	LED[3:0]	Signal to control LED[3:0]; Display LFSR state.

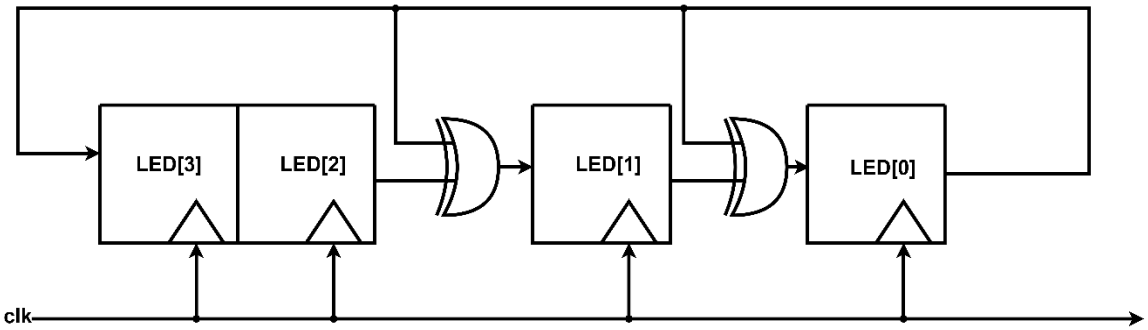
b. **FSM Specification**

- The FSM consists of three states:
  1. **IDLE**: Password generation using the 4-bit LFSR
  2. **PLAY**: User attempts to enter password by BTNL and BTNR
  3. **FINAL**: If success, the FSM enters the FINAL state

  At any time, pressing **rst** resets the FSM to **IDLE**, resets the LFSR to **4'b1001**, resets **out[1:0]** to **2'b00**, and clears **num[3:0]** to **4'b0000**.

IDLE state

- Implement the following **4-bit LFSR** to generate a pseudo-random password:



- The state of the LFSR is displayed on LED[3:0].
- Pressing BTND shifts the LFSR one cycle to the right.
  - E.g., 1001→1111→1100→0110→0011→1010→0101→0101→1001
- The LFSR state serves as the **password**.
- Press BTNU to move the **PLAY** state.

PLAY state

- Pressing BTND resets **num[3:0]** to 4'b0000.
- Password input:
  - BTNL inputs 1
  - BTNR inputs 0
- Each input shifts into the LSB (num[0]) of num[3:0] in a **left-shifting** manner.
- The number num[3:0] is displayed on LED[15:12].
- If the entered sequence matches the password:
  - FSM transitions to the **FINAL** state
- If the sequence does not match, out[1:0] remains 2'b00.
- After 4 inputs, additional presses of BTNL or BTNR have no effect until BTND is pressed to reset input.

FINAL state

- In this state, the correct password has been entered.
- num[3:0] = 4'b0000 (LEDs [15:12] are turned off).
- out[1:0] = 2'b11 (i.e., LED[8:7] ON)
- Pressing BTNU returns FSM to IDLE, where a new password can be generated.

Example Operation

- Assume password = 4'b1001:
- Start in IDLE → LFSR shows 1001 on LED[3:0].

- Press BTNU → move to PLAY state.
- Enter inputs:
  - Press BTNL → sequence becomes 0001 (LED[15:12] = 0001)
  - Press BTNR → sequence becomes 0010 (LED[15:12] = 0010)
  - Press BTNR → sequence becomes 0100 (LED[15:12] = 0100)
  - Press BTNL → sequence becomes 1001 (LED[15:12] = 1001)
- FSM detects correct password → FSM enters FINAL state.
- In FINAL state:
  - num[3:0] = 0000 (LED[15:12] lights OFF)
  - out[1:0] = 11 (LED[8:7] lights ON)
- Press BTNU → return to IDLE for a new password.

Demo video:

- Please refer to the attached MP4 file.

Grading Breakdown

Functionality	**Weight (100%)**
IDLE state: LED[3:0] can be controlled by LFSR	25%
PLAY state: Inputting number by BTNL and BTNR, and the number can be shown on num[3:0] and shifted correctly.	30%
PLAY state: BTND can reset the input.	20%
PLAY state: The FSM can detect the password correctly, and the result can be shown on LED[8:7] correctly.	10%
FINAL state, BTNU back to IDLE state	10%
rst	5%

Hint

- You must use the template we provide for your design. (lab4_basic, debounce, one_pulse)
- We will provide you with the constraints file.

Attention

- Download the *.cpp files from OJ. Change them to *.zip and decompress them. (Skip the *.h files. The OJ enforces that there must be a *.h file.)
- Submit the Verilog file to OJ immediately once it is done.
- Please take the OJ password slip with you when leaving your seat. Do not litter!
- If you create several modules, merge them into a single Verilog file. Submit only one Verilog file, lab4_basic.v.
  - You have the responsibility to check if the submission is successful.
  - DO NOT submit ZIP files, which will be considered as incorrect format.
- DO NOT copy-and-paste code segments from PDFs (hidden characters may cause hard-to-debug syntax errors).